

AWS IAM (Identity and Access Management)

00-foundations/iam

Overview

IAM is the authorization layer that controls who can call AWS APIs and what they are allowed to do after they authenticate. It is the first security boundary for every workload you deploy in AWS.

In a machine learning environment, IAM protects datasets, training infrastructure, model artifacts, and deployment pipelines. If IAM is weak, the rest of the platform is weak no matter how good the model is.

Core Concepts

- IAM identities include users, groups, and roles. Human operators increasingly use federation or SSO, while workloads should use roles with temporary credentials instead of long lived access keys.
- Policies are JSON documents that define allowed or denied actions on resources, optionally constrained with conditions such as tags, source VPC endpoints, time, or MFA state.
- AWS evaluates identity based policies, resource policies, permission boundaries, session policies, and organization controls together. Explicit deny always overrides an allow.

How It Works

- A principal signs a request with credentials that represent a user or an assumed role. AWS then evaluates all applicable policies before the target service processes the request.
- Service roles are attached to workloads such as Lambda functions, EC2 instances, or SageMaker training jobs so the code can access S3, logs, and secrets without embedding credentials.
- Cross account access is commonly implemented by allowing one account to assume a role in another account. That is a standard pattern for promoting models from development to production.

AI and ML Relevance

- Training jobs often need read access to raw data, write access to model artifacts, and permission to emit logs and metrics. Those permissions should be separated from deployment permissions.
- A mature ML platform uses different roles for experimentation, scheduled pipelines, model registry operations, and production inference so blast radius stays small.

Operational Notes

- Prefer least privilege, short lived credentials, and role assumption over static keys. Long lived access keys in notebooks or scripts are a recurring source of compromise.
- Use CloudTrail and IAM Access Analyzer to review who used which permissions and where policies are broader than necessary.
- Separate human admin access from application access. People troubleshoot systems; workloads should have narrow machine oriented permissions.

What To Remember

- Authentication answers who you are. Authorization answers what you can do.
- Roles are the normal answer for AWS workloads. Users with keys should be the exception.
- If something is mysteriously denied, look for an explicit deny, a missing resource scope, or an organization level control.